

Realistic Enemy AI in Stealth Games

Marta Pecharromán, Ander Fernández, and Suyay Charles

1. Introduction

Regarding game AI, it is common to find numerous pathfinding algorithms to enable characters to navigate 3D or 2D environments; Dijkstra, A*, JPS, Bellman-Ford.... These are the basis for the behavior an agent in Stealth Games, or games in which enemies seek and chase the player. Nonetheless, it is not realistic enough for the enemies to brainlessly chase you, without caring about external reasons, like other enemies near them, possible hiding spots for the player, acting in groups... This paper aims to solve several aspects which will make the navigation more realistic and optimal.

To begin with, how do we simulate good group navigation? With group navigation or crowd movement, we refer to real groups of people walking down the street, soldiers patrolling a campsite, or hordes of zombies wandering. These individuals, should not be intersecting each other in the exact same path, and they should follow a set of rules which make their movement look ordered, similar to the real world. We will discuss how we get this thanks to flocking and social group behaviors.

On the other hand, crowd movement in games is often managed through predetermined paths and adjustments to avoid collisions. As the number of agents increases and their destinations become similar, it becomes ineffective. Each agent tries to stick to its path, disregarding alternative paths and resulting in a struggle against the rest of the crowd. With congestion maps, we will make agents take decisions depending on the position their mates are in, taking paths that are less crowded to try to catch the player in a smarter way. Also we will show how we could use Flow Fields to optimize maps with lots of enemies.

Finally, we will use techniques described above to analyze Optimal Cover Placement, which in game design, ensures players find safe spots to avoid detection and rest, enhancing their overall experience. By analyzing factors like openness, enemy congestion, and visibility, an algorithm identifies the best cover locations. This approach helps level designers balance intense and calm moments, maintaining engaging and enjoyable gameplay. Various techniques, including pathfinding algorithms, are used to navigate these covers while considering enemy vision and congestion, enriching the strategic experience for players.

2. Group Behaviors

In general navigation simulation literature, we find three main approaches to group navigation: flocks, formations and social groups. Let's define them before choosing the most appropriate for our objective.

2.1. Flocking

A flock, in nature, is what we call a group of birds or a school of fish in movement. In AI we can use flocking to simulate movement to other species as well as humans. Every entity in a flock travels with a similar speed and direction, creating a cohesive group without a strict movement. In a flock of birds for example, they are not facing the same direction, neither they are evenly distributed, but they follow some rules inside the group.

2.2. Formations

Contrarily to flocks, formations are groups in which the rules are very strict, and lead to more even and balanced arrangements. Formations can be very useful in situations which require the agents to face the same direction, as in an army shooting a target.

2.3. Social Groups

In real life, most entities don't move as synchronized as fish or graceful as birds, and neither they move as strict as an army attacking a base. Instead, we have our own set of rules that define our behavior as a group: enable dialogue between the group, reconfigure depending on obstacles, splitting and merging, following the leader of the group.

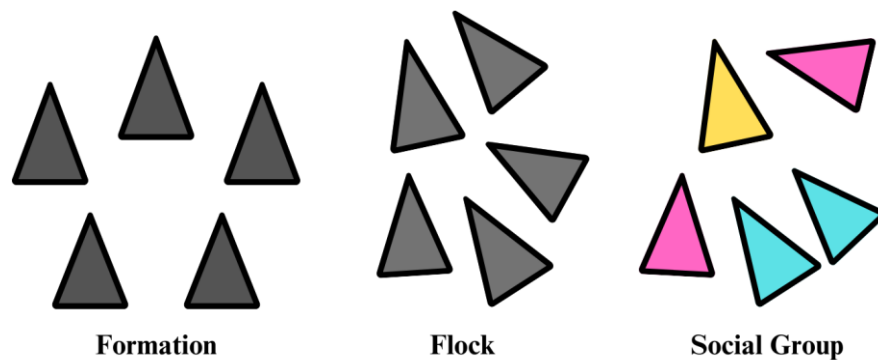


Figure 2.1

2.4. Problem Statement

For our purpose, that might be too hard (and unnecessary) to simulate a social group that exactly resembles real life. If we want to create a scenario with a large amount of human enemies, we should be avoiding heavy computations. Then, what could be the approach to get a realistic crowd simulation, which also reduces computational cost?

2.5. Solution

Contemplating a natural behavior of a human we discard a movement based on Formations. Human movement reminds more of a flock, acting like social groups too.

In our approach, we combined both methods to create crowds that follow a hierarchy, giving them a set of rules, defining their behavior.

2.6. The Group

We based the roots of our flocking algorithm in Craig Reynold's Boid model.

2.7. Boids

The model consists of three simple steering behaviors which describe how each boid (each individual in the flock) maneuvers based on the position and velocities of nearby flock mates.

Alignment: aligns a boid's direction averaging the directions of the velocity of other boids.

Cohesion: averages the position of the boids to make them steer toward the center.

Separation: gets direction vector from other boids to separate them avoiding congestion.

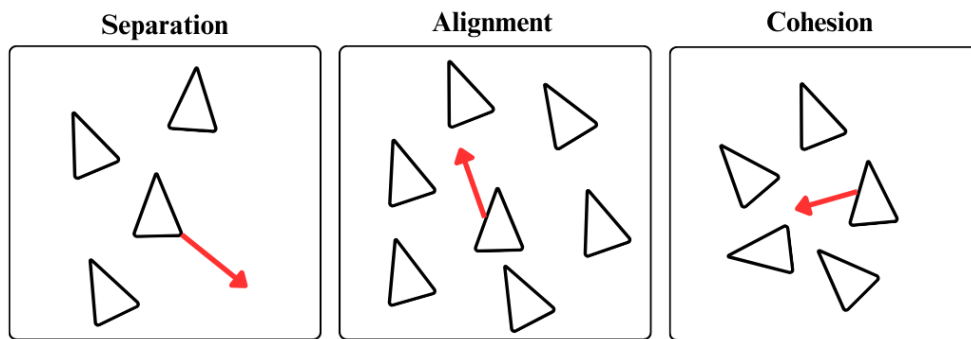


Figure 2.2

2.8. Additional Forces and Adjustments

Unfortunately, this model alone didn't fit our desired movement for the agents, as it was too fast and chaotic for what we picture the movement of a human.

Obstacle Avoidance Force: A subtle separating force coming for the closest wall, avoiding going through walls and clipping them.

Inertia Force: Stabilizes the steering from the flock getting rid of 'randomness'. It is calculated giving weight to current boid's velocity vector.

Force and Speed limiting: All forces are normalized and added after. But they can lead to very strong forces that abruptly push a boid losing realism, so we clamp the values.

Leader Seeking: The leader of the group has a stronger weight in the group so the group follows the leader's direction. More about the leader is discussed in the following section.

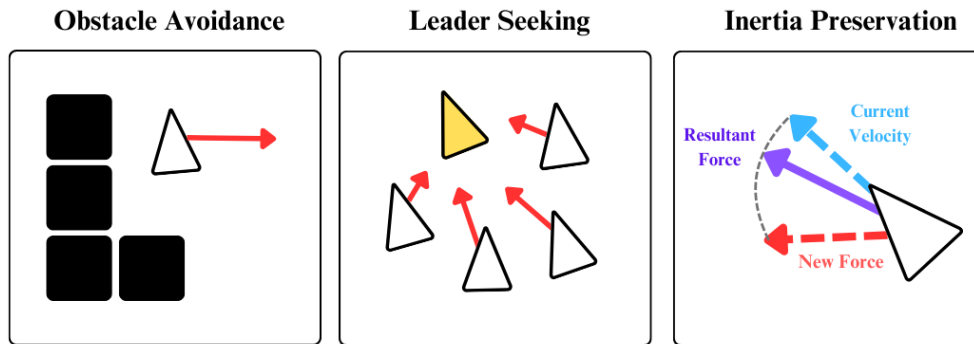


Figure 2.3

All these forces are normalized and clamped, and then added together resulting on a new direction and orientation for the agent.

On top of that, we also alter the separation weight slightly over time to give more variation and dynamism to the flock.

2.9. Social Group Hierarchy

To manage the Crowds, we decided to apply a simple two level hierarchy featuring a Leader and its followers. The leader is in charge of the whole group's navigation, and it will be the only one performing pathfinding, a single query is enough for the whole group, reducing considerably computational cost, as its followers will keep on the pack using cheaper math.

Entities will check if they are close enough for another entity to create a group, if so, one of them will become a leader. When more individuals approach the group, they will become followers.

In the case that another group approaches the horde, the groups will merge, taking as leader the one with the largest number of followers. While in the group, as mentioned before, the leader will navigate through the map, either patrolling, seeking, chasing... Without any flock force being applied to it. On the other hand, the followers will just be displaced and orientated depending on the tendency of the rest of the flock and specially the leader, who has a greater influence.

As we don't want the agents to move very rigidly, we gave the flock subtle forces which

sometimes can make them clip a wall and be put off the horde. To prevent that, leaders have a second radius bigger than the one in charge of detecting new followers: when a boid gets out of the inner radius but it is still inside the second (in between both), it will perform a short pathfinding request to its leader's position. As they still should be very close, it won't add much overhead cost. And it is also very convenient for obstacle avoidance. Finally, if an agent gets out of the outer radius, because the previous pathfinding wasn't enough to catch up to the group, it will start to patrol again on its own.

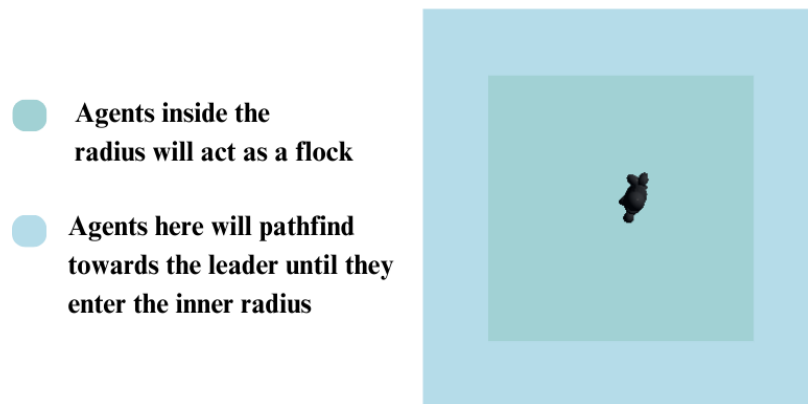


Figure 3.4

3. Congestion Map and Flow Field

The shortest path may not always be the best choice when trying to translate to a certain location. Moving across a crowd could take more time than picking a less crowded path. Is preferable to take a longer path if the time to reach the goal position is less than if we choose the ideal path (the shortest one), or to spread the search making enemies more separated.

3.1. Solution

Congestion maps are used to handle that situation. They store the information of the aggregate crowd density in the current frame. It is used in the pathfinding algorithm to discourage movement through crowded areas.

3.2. Congestion Map

To compute the congestion map, we will have some values that will be added to the cells that surround the enemies. Only the cells that are at a certain distance will be affected by the congestion of that enemy. After all the computations we will normalize the congestion map dividing all the cells by the highest value in the map. The values will go from 0 to 1, being 1 the most congested area of the grid, and 0 the areas without congestion.

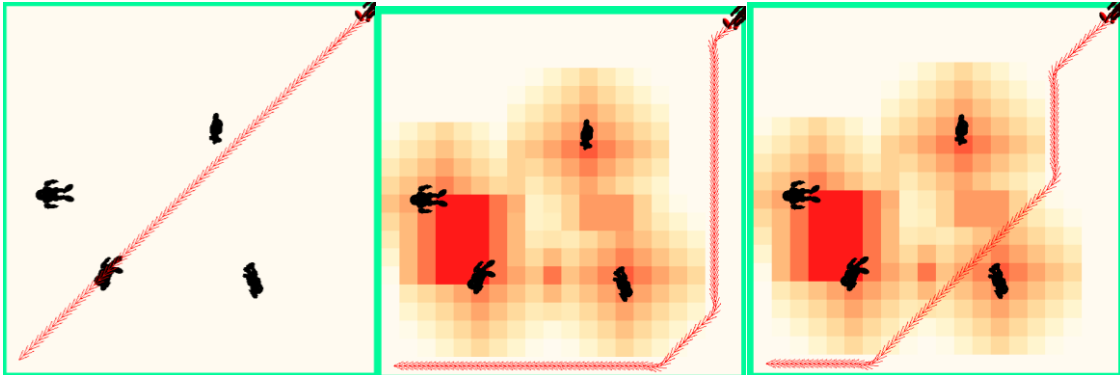


Figure 3.1

Figure 3.2

Figure 3.3

The *Figure 3.1* shows the ideal path from the player to the position (0,0) in the grid. There is an enemy in the path, which may slow the travel.

The *Figure 3.2* shows an alternative path which considers the congestion of the enemies in a certain area. Depending on the weight of the congestion (how much affects to the choice of the path) the path will vary (*Figure 3.3*).

3.3. Flow Field

Flow fields provide better performance when dealing with high amounts of enemies in a scene. When a lot of agents try to go to a goal without using a flow field each of the agents has to compute the path that he must follow. When using flow fields, we compute the cost field and integration field before getting the final flow field. It is only recomputed when the player does not have the same position of the previous frame. The process of creating a vector flow field is subdivided into three sub processes.

3.3.1. Cost field

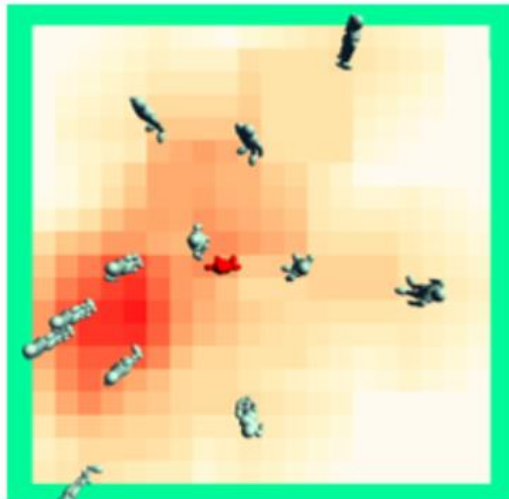
The previously calculated congestion map will act as the cost field for the vector flow field. When computing the vectors of each cell in the flow field we will consider these values to avoid the agents to follow the same paths to reach the same goal. It is important that it is static, because if it changes dynamically (congestion map) the flow field would have to be recalculated each frame which would lead to worse performance.

3.3.2. Integration field

It is where most of the work in the flow field computation is done. The pathfinding algorithm mostly used for it is Dijkstra. It represents the cumulative minimum cost to reach a target or goal from each cell in the grid. It is calculated with the cost field previously calculated to compute each path avoiding the crowded paths.

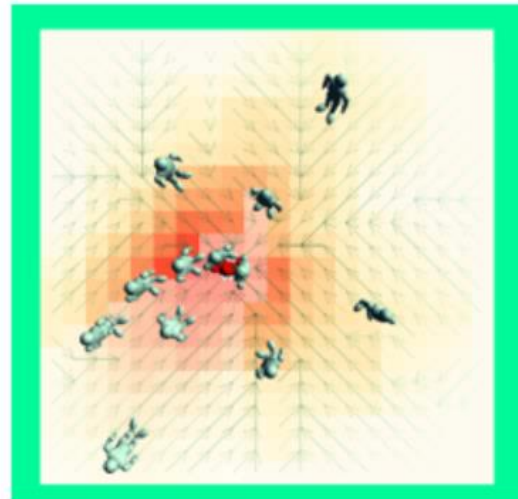
3.3.3. Flow field

With the integration field we can calculate the vector flow field. Each vector in it points towards the neighbor that is closest to the goal.



Cost field

Figure 3.4



Flow field

Figure 3.5

4. Optimal cover placement

With optimal cover placement we refer to the place where the player is less prone to be caught by an external agent. Also, these places serve as resting areas for the user.

4.1. Game experience and level design

Game experience or user experience is the overall experience the player has while interacting with a videogame. The goal of this term is to create an enjoyable and immersive experience.

Traditionally, UX design is about feeding the needs of the users. To achieve this, UX designers must do a good job when building the level, by using a good pacing. A good pacing or flow is built with intense and calm moments, if this is well balanced the player will be engaged and entertained.

In order to help the level designer, we have created an algorithm that calculates the best cover spots along the entire map.

4.2. Creating an optimal cover map

To create covers, we have need to create several layers that represent different types of hazard and secure points. Starting with the openness, this metric is used to point out the cells on map that are closer to a wall. Usually being close to a wall or corner means to be safer as you are protected by the near wall as seen in *Figure 4.1*.

In *Figures 4.2 and 4.3*, we can see the areas that produce peak action moments. *Figure 4.2* represents the congestion of the enemies. If foes gather the congestion layer at those met points will have huge values that mark how dangerous that zone can be.

In *Figure 4.3*, we have a layer that shows the perception and sight areas of the enemies. These areas are a major source of danger that we want to manage while designing the levels. The perception is a radius, and the sight is the FOV of the enemy agent.

The last layer we use is the visibility layer. This metric is the one with the highest weight over the optimal cover layer. The visibility layer is calculated by counting all the number of cells that are visible to the cell being computed. The result is a map where the zones with higher risk/hazard have a bigger value than the safer zone that conversely have lower values.

Considering all these layers, we are able to compute the optimal cover layer, see *Figure 4.5*. The value of each cell on the optimal cover value is computed the following way:

$$\text{coverValue} = \text{clamp}(\text{visibility} + \text{enemiesVision} + \text{congestion} - \text{openness}, 0.0f, 1.0f);$$

The result of the optimal cover value layer is what needs to be analyzed by a UX or level designer. The interesting part of this is to see if the pacing of the level is pleasant and does not lead to trough situation that leads to tedious gameplay or overexciting situations that will make the player bored of doing the same action repeatedly.

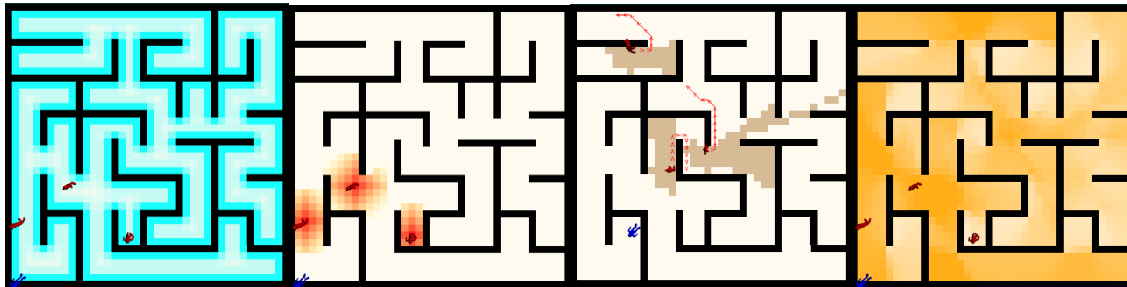


Figure 4.1

Figure 4.2

Figure 4.3

Figure 4.4

4.3. Finding an optimal cover

Numerous techniques will bring us to the best cover. We will briefly discuss some of them.

The first approach is to travers all the layer and after finding the best cover make the player go directly to this one. It is simple but you do not take the enemies perception and sight areas into account. Refer to *Figure 4.6* for an example.

The second approach, similar to the first one, will go for the best cover but it would have the perception and sight zones considered. We will make use of the Dijkstra's algorithm and taking advantage of the layer we can use it as a weighted graph so that the path to reach the best cover does not fall into the enemy's sight. This approach may block the path of the player ass seen in *Figure 4.3* where half the map is blocked by the foe's vision.

As the third and last technique we are going to take advantage of the congestion layer and although we may fall into the enemy's sight we are going to try not to fall into their hands. This technique is explained previously in the document at chapter 3.3.2

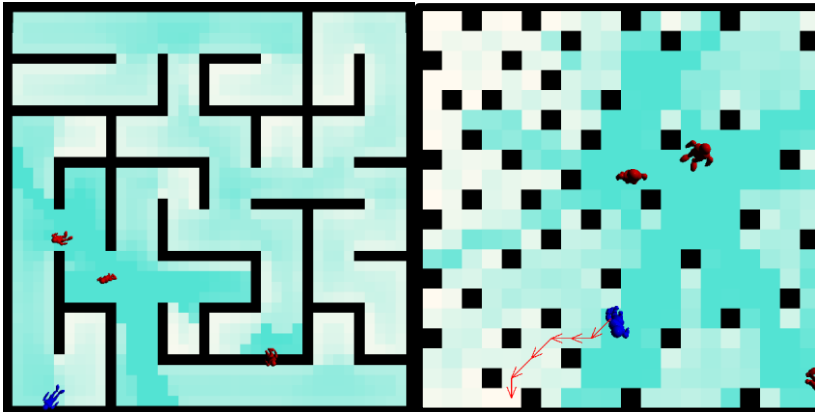


Figure 4.5

Figure 4.6

5. Conclusion

This paper addresses the challenges of simulating realistic and optimal navigation for agents in video games, particularly in stealth scenarios. Traditional pathfinding algorithms alone are not enough for creating believable behaviors where enemies consider group dynamics, environmental obstacles, and player strategies.

We explored group navigation methods, identifying flocking and social group behaviors as the most suitable. By refining Craig Reynolds' Boid model, we achieved human-like movement through additional forces and hierarchical structures, enhancing visual realism and reducing computational costs by assigning pathfinding tasks to group leaders.

To improve navigation further, we introduced congestion maps and flow fields. Congestion maps provide real-time crowd density data, allowing agents to avoid crowded areas, while flow fields offer precomputed navigation solutions, ensuring smooth and efficient movement for large groups.

Additionally, we focused on optimal cover placement, which significantly impacts player experience. By layering metrics like openness, enemy congestion, and visibility, we developed an algorithm to identify the safest and most strategic cover spots. This system helps level designers in balancing intense and calm moments, enriching the game's strategic depth.

In conclusion, the integration of advanced group navigation techniques, congestion-aware pathfinding, and optimal cover placement enhances AI behavior in games. These innovations create more immersive and strategically challenging environments, contributing to a richer and more dynamic player experience.

6. References

[Reynolds 86] Craig W. Reynolds. 1986 *Boids Background and Update*.

<https://www.red3d.com/cwr/boids/>

[Reynolds 86] Craig W. Reynolds. 1986. *Flocks, Herds, and Schools. A Distributed Behavioral Model*. Symbolics Graphics Division

<https://leifnode.com/2013/12/flow-field-pathfinding/>

Jamie Snape, Stephen J. Guy, Deepak Vembar, Adam Lake, Ming C. Lin, and Dinesh Manocha. 2012. *Reciprocal Collision Avoidance and Navigation for Video Games*.

[Chanut 21] Clodéric Mars and Jérémy Chanut. 2021. *Game AI Pro*. Chapter 20. Hierarchical Architecture for Group Navigation Behaviors.

[Welsh 21] Rich Welsh. 2021. *Game AI Pro*. Chapter 27. Looking for Trouble. Making NPCs Search realistically.

[Botta 21] Mark Botta. 2021. *Game AI Pro*. Chapter 33. Infected AI in The Last of Us.

[Pemmaraju 13] Vijay Pemmaraju. 2013. *3 Simple Rules of Flocking Behaviors: Alignment, Cohesion and Separation*.

[Emerson 21] Elijah Emerson 2021. *Game AI Pro*. Chapter 23. Crowd pathfinding and steering using flow field tiles

[Pentheny] Graham Pentheny 2021. *Game AI Pro*. Chapter 17. Advanced techniques for robust, efficient crowds.

[Shi 15] Yinxuan Shi, Roger Crawfis. 2015. Optimal Cover Placement Against Static Enemy Positions.

https://www.researchgate.net/publication/269723593_Optimal_Cover_Placement_Against_Static_Enemy_Positions